Structures

Introduction.....1
Why Do We Need Structures?.....2
Syntax of Structure.....2
Structure Diagram.....3
Data Members.....3
Creating Structure Variables (Objects).....4
Accessing Data Members.....4
Taking Input from User.....6
Member Functions.....7
Structure with Data Members and Member Function.....8
Member Function for Input and Output.....9
Structure Initialization.....11
Array of Structures.....11
Nested Structure.....13
Pointer to Structure.....14
Memory Representation.....15
Difference Between Data Member and Member Function.....16
Difference Between Structure and Class.....16
Advantages of Structures.....16
Practice Questions.....17

Introduction

A **structure** is a **user-defined data type** in C++ that groups variables of different data types into a single unit.

It allows us to store related information together.

Example

A student has

- Roll Number
- Name

- Age
- Marks

Instead of declaring four separate variables, we can combine them into one **Student structure**.

Definition

> **A structure is a collection of related variables (called data members) and functions (called member functions in C++) grouped together under a single name.**

Unlike C, **C++ structures can contain both data members and member functions.**

Why Do We Need Structures?

Suppose we want to store information of 100 students.

Without structure

```
int roll1, roll2, roll3;
char name1[30], name2[30], name3[30];
float marks1, marks2, marks3;
```

The program becomes difficult to manage.

Using structure

```
Student s1, s2, s3;
```

Each object stores complete information about one student.

Syntax of Structure

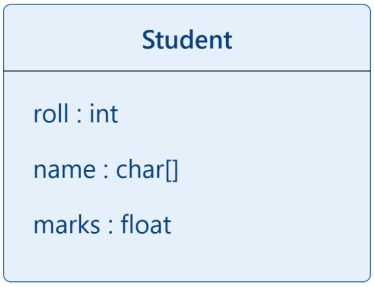
```
struct StructureName
{
   // Data Members

   // Member Functions
};
```

Example

```
struct Student
{
   int roll;
   char name[30];
   float marks;
};
```

Structure Diagram



Each variable declared using Student gets its own copy of these members.

Data Members

Variables declared inside a structure are called **Data Members**.

Example

```
struct Student
{
    int roll;          // Data Member

    char name[30];     // Data Member
```

```
    float marks;       // Data Member
};
```

Explanation

Here,

- roll
- name
- marks

are called **Data Members** because they store data.

Creating Structure Variables (Objects)

Student s1;

Student s2;

Student s3;

Each object contains its own copy of all data members.

s1
---
roll
name
marks

s2
---
roll
name
marks

Accessing Data Members

Use the **dot (.) operator**.

Program

```
#include<iostream>
#include<cstring>
using namespace std;

// Define a structure named Student
// Structure groups different types of data under one name
struct Student
{
    int roll;        // Store student's roll number
    char name[30];   // Store student's name
    float marks;     // Store student's marks
};

int main()
{
    // Create a structure variable 's' of type Student
    Student s;

    // Assign roll number to the structure variable
    s.roll = 101;

    // Copy the string "Rahul" into the name array
    // strcpy() is used to copy one string into another
    strcpy(s.name, "Rahul");

    // Assign marks to the structure variable
    s.marks = 91.5;

    // Display the roll number
    cout << "Roll = " << s.roll << endl;

    // Display the student's name
    cout << "Name = " << s.name << endl;
```

```
    // Display the student's marks
    cout << "Marks = " << s.marks;

    return 0;   // Indicate successful program termination
}
```

Output
Roll = 101
Name = Rahul
Marks = 91.5

Taking Input from User

```
#include<iostream>
using namespace std;

// Define a structure named Student
// Structure is used to group related data members
struct Student
{
    int roll;       // Store student's roll number
    string name;    // Store student's name
    float marks;    // Store student's marks
};

int main()
{
    // Create a structure variable 's' of type Student
    Student s;

    // Ask the user to enter the roll number
    cout << "Enter Roll Number : ";
    cin >> s.roll;

    // Ask the user to enter the student's name
    cout << "Enter Name : ";
    cin >> s.name;
```

```
    // Ask the user to enter the student's marks
    cout << "Enter Marks : ";
    cin >> s.marks;

    // Display a heading before printing student details
    cout << "\nStudent Details\n";

    // Display the roll number
    cout << "Roll : " << s.roll << endl;

    // Display the student's name
    cout << "Name : " << s.name << endl;

    // Display the student's marks
    cout << "Marks : " << s.marks;

    return 0;   // Indicate successful program termination
}
```

Member Functions

A function declared inside a structure is called a **Member Function**.

Member functions operate on the data members of the structure.

Example

```
struct Student
{
    int roll;
    float marks;

    void display()
    {
        cout<<roll<<endl;

        cout<<marks<<endl;
    }
};
```

Here, display() is a **Member Function**.

Structure with Data Members and Member Function

```
#include<iostream>
using namespace std;

// Structure named Student
struct Student
{
    // Data members of the structure
    int roll;       // Stores the student's roll number
    string name;    // Stores the student's name
    float marks;    // Stores the student's marks

    // Member function to display student details
    void display()
    {
        // Display the roll number
        cout << "Roll Number : " << roll << endl;

        // Display the student's name
        cout << "Name : " << name << endl;

        // Display the student's marks
        cout << "Marks : " << marks << endl;
    }
};

int main()
{
    // Create an object 's' of the Student structure
    Student s;

    // Assign values to the data members of object 's'
    s.roll = 101;
```

```
    s.name = "Amit";
    s.marks = 95.5;

    // Call the display() member function using the object
    s.display();

    return 0;   // Indicate successful program termination
}
```

Output
Roll Number : 101
Name : Amit
Marks : 95.5

Member Function for Input and Output

```
#include<iostream>
using namespace std;

// Define a structure named Student
struct Student
{
    // Data members of the structure
    int roll;        // Stores the student's roll number
    string name;     // Stores the student's name
    float marks;     // Stores the student's marks

    // Member function to take student information from the user
    void input()
    {
        // Ask the user to enter the roll number
        cout << "Enter Roll : ";
        cin >> roll;

        // Ask the user to enter the student's name
        cout << "Enter Name : ";
        cin >> name;
```

```
        // Ask the user to enter the student's marks
        cout << "Enter Marks : ";
        cin >> marks;
    }

    // Member function to display student information
    void display()
    {
        // Display a heading
        cout << "\nStudent Information\n";

        // Display the roll number
        cout << "Roll : " << roll << endl;

        // Display the student's name
        cout << "Name : " << name << endl;

        // Display the student's marks
        cout << "Marks : " << marks << endl;
    }
};

int main()
{
    // Create an object 's' of the Student structure
    Student s;

    // Call the input() function using the object
    // This takes student information from the user
    s.input();

    // Call the display() function using the object
    // This displays the entered student information
    s.display();

    // Indicate successful program termination
    return 0;
}
```

This is one of the best examples to explain **data members** and **member functions**.

Structure Initialization

```cpp
#include<iostream>
using namespace std;

// Define a structure named Student
struct Student
{
    int roll;        // Stores the student's roll number
    string name;     // Stores the student's name
    float marks;     // Stores the student's marks
};

int main()
{
    // Create a Student structure object 's'
    // Initialize its members directly with values
    Student s = {101, "Rahul", 89.5};

    // Display the roll number
    cout << s.roll << endl;

    // Display the student's name
    cout << s.name << endl;

    // Display the student's marks
    cout << s.marks;

    return 0;   // Indicate successful program termination
}
```

Array of Structures

```cpp
#include<iostream>
using namespace std;
```

```cpp
// Define a structure named Student
// Structure is used to group related data members
struct Student
{
    int roll;       // Store student's roll number
    string name;    // Store student's name
    float marks;    // Store student's marks
};

int main()
{
    // Create a structure variable 's' of type Student
    Student s;

    // Ask the user to enter the roll number
    cout << "Enter Roll Number : ";
    cin >> s.roll;

    // Ask the user to enter the student's name
    cout << "Enter Name : ";
    cin >> s.name;

    // Ask the user to enter the student's marks
    cout << "Enter Marks : ";
    cin >> s.marks;

    // Display a heading before printing student details
    cout << "\nStudent Details\n";

    // Display the roll number
    cout << "Roll : " << s.roll << endl;

    // Display the student's name
    cout << "Name : " << s.name << endl;

    // Display the student's marks
    cout << "Marks : " << s.marks;

    return 0;   // Indicate successful program termination
}
```

Nested Structure

```
#include<iostream>
using namespace std;

// Define a structure named Date
// It stores day, month and year
struct Date
{
    int day;        // Store the day
    int month;      // Store the month
    int year;       // Store the year
};

// Define a structure named Student
struct Student
{
    int roll;       // Store student's roll number
    string name;    // Store student's name

    // Create a Date structure variable inside Student
    // This is called a nested structure
    Date dob;       // Store student's date of birth
};

int main()
{
    // Create a structure variable 's' of type Student
    Student s;

    // Assign roll number
    s.roll = 101;

    // Assign student's name
    s.name = "Rahul";

    // Access the Date structure through the 'dob' member
```

```
    // and assign the day, month and year
    s.dob.day = 15;
    s.dob.month = 8;
    s.dob.year = 2004;

    // Display the student's name
    cout << s.name << endl;

    // Display the date of birth
    cout << s.dob.day << "/";

    // Display the month
    cout << s.dob.month << "/";

    // Display the year
    cout << s.dob.year;

    return 0;   // Indicate successful program termination
}
```

Pointer to Structure

```
#include<iostream>
using namespace std;

// Define a structure named Student
struct Student
{
    int roll;        // Stores the student's roll number
    string name;     // Stores the student's name
};

int main()
{
    // Create an object 's' of the Student structure
    Student s;

    // Create a pointer to the Student structure
```

```
    Student *ptr;

    // Store the address of object 's' in the pointer
    ptr = &s;

    // Access and assign the roll member using the structure pointer
    // The -> operator is used to access structure members through a
pointer
    ptr->roll = 101;

    // Access and assign the name member using the structure pointer
    ptr->name = "Amit";

    // Display the roll number using the structure pointer
    cout << ptr->roll << endl;

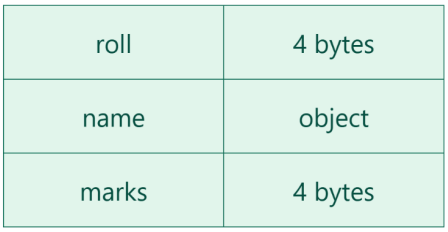
    // Display the student's name using the structure pointer
    cout << ptr->name;

    return 0;   // Indicate successful program termination
}
```

The arrow operator (`->`) is used with pointers.

Memory Representation

Student s



Every object has its own copy of the data members.Member functions are **not duplicated** for every object. They are shared by all objects.

Difference Between Data Member and Member Function

Data Member	Member Function
Stores data	Performs operations
Variable	Function
Occupies memory in every object	Code is shared by all objects
Accessed using dot operator	Called using dot operator
Example: roll	Example: display()

Difference Between Structure and Class

Structure	Class
Default access is public	Default access is private
Used mainly for grouping related data	Used for Object-Oriented Programming
Can have data members	Can have data members
Can have member functions	Can have member functions

Advantages of Structures

- Groups related data together.
- Makes programs easy to understand.
- Reduces the number of variables.
- Supports member functions in C++.
- Can create arrays of structures.
- Can create nested structures.
- Easy to pass to functions.

Practice Questions

Q1. What is a structure?

A structure is a user-defined data type used to group related variables and functions.

Q2. What are data members?

Variables declared inside a structure.

Q3. What are member functions?

Functions declared inside a structure.

Q4. Which operator is used to access members?

Dot operator (`.`)

Q5. Which operator is used with structure pointers?

Arrow operator (`->`)

Q6. What is the default access specifier of a structure?

public

Q7. Can structures have functions in C++?

Yes. Structures in C++ can contain member functions, constructors, destructors, and even support inheritance.